Tmux Is All You Need

A Terminal-Multiplexer Architecture for Recursive Coding-Agent Orchestration

Fabio Pauli[1]

apritzclass@gmail.com

Abstract

The dominant paradigm for supervising autonomous coding agents is *recurrent*: a single human operator context-switches between one agent at a time, carrying task state forward in their own working memory, dispatching instructions in sequence. This design is inherently sequential, which precludes parallelization across agents and makes the human the bottleneck and the single point of failure. We propose the *Tmux Orchestration Architecture*, a control substrate based solely on the terminal multiplexer `tmux`, dispensing with recurrence in the human entirely. In our architecture an *orchestrator* agent attends directly to any worker agent in the swarm through a small set of content- and address-based primitives — `send-keys`, `capture-pane`, `pipe-pane`, hooks, and `wait-for` — reducing the maximum path length between any two agents from $O(n)$ sequential human operations to $O(1)$ addressable operations. Because a tmux server persists independently of any attached client, and because a pane can itself run a tmux client, the architecture is *recursive*: an orchestrator can spawn sub-orchestrators, forming a tree of coordination while keeping the human attached at the root, in the loop, *eventually*. We show that tmux orchestration is superior to recurrent human management in three respects — total control operations per step, number of required sequential human interventions, and maximum inter-agent path length — and describe the primitives, the topology, and the control loop in enough detail to reproduce a semi-autonomous, human-supervised agent swarm.

1 Introduction

Recurrent supervision of coding agents — Claude Code, Codex, and similar LLM-driven development tools — has firmly established itself as the state of the art in single-operator workflows. The operator opens one agent, states a task, waits, reads the result, forms the next instruction, and repeats. State is carried forward as a hidden representation h_t inside the operator's own head,

$$h_t = f(h_{t-1}, x_t),$$

where x_t is the t-th agent interaction and h_t is the operator's evolving mental model of the project. This recurrent formulation, in its various guises, has remained the boundary of what a single human can manage. It has two fundamental limitations, both familiar from the recurrent sequence-modeling literature [3, 1]:

1. **Sequential computation.** The operator can advance only one interaction per step. Two agents cannot be truly supervised at once; attention is time-sliced, not parallel. Throughput is bounded by human cycle time, not by the number of available agents or cores.
2. **Path length.** To route a result produced by agent A into the working context of agent B, the operator must read A, hold the result in working memory, context-switch, and re-type it to B. Connecting any two of n agents costs $O(n)$ sequential human operations, and every hop passes through the single-threaded hidden state h_t. Long dependency chains degrade the representation exactly as long-range dependencies degrade an RNN.

Attention mechanisms [1] dissolved the analogous problem in sequence modeling by allowing any position to be reached from any other in a constant number of operations, and the Transformer [4] showed that once you have attention, *recurrence is unnecessary*. We make the equivalent claim for agent orchestration. We propose the *Tmux Orchestration Architecture*, a model architecture eschewing human recurrence and relying entirely on a terminal multiplexer to draw global dependencies between agents. Tmux allows significantly more parallelization; a single orchestrator can drive a swarm of coding agents, routing outputs to inputs directly, with the human elevated from the inner loop to a supervisory gate.

The contributions of this paper are: a mapping from the components of self-attention onto concrete tmux primitives (§3), including scaled selection (§3.2.1), multi-head orchestration (§3.2.2), and positional encoding via the session/window/-pane address space (§3.5); a "Why Tmux" analysis (§4) comparing orchestration substrates by control cost, required human interventions, and inter-agent path length; and a description of how to deploy, batch, schedule, and regularize a live swarm (§5), including the human-in-the-loop gate as an explicit regularizer.

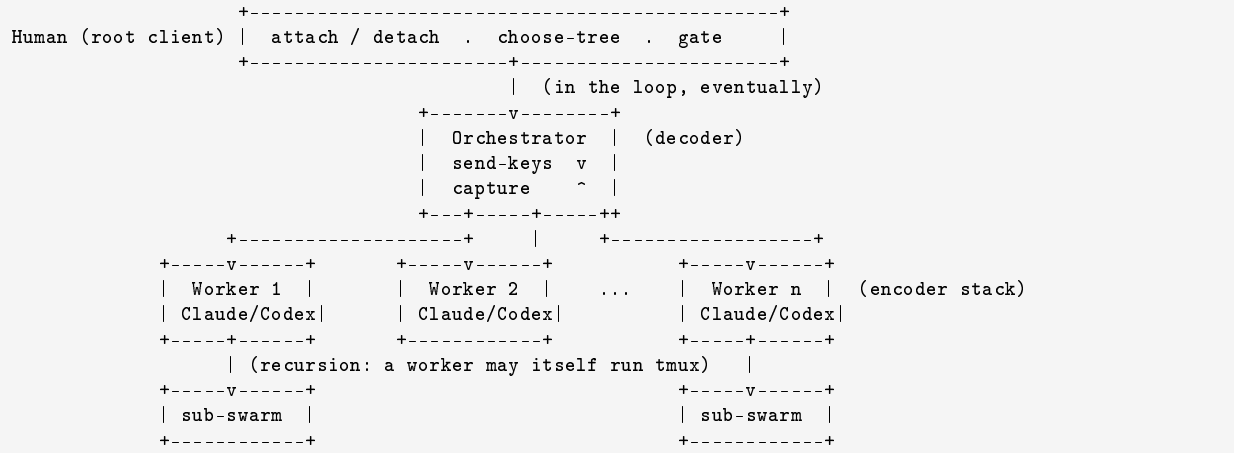

Figure 1: The Tmux Orchestration Architecture. An orchestrator (decoder) attends over a stack of worker panes (encoder) via `send-keys`/`capture-pane`; workers may recurse into sub-swarms; the human attaches at the root as a supervisory gate.

2 Background

The goal of reducing sequential human involvement forms the basis of numerous tools: shell job control, `make -j`, CI runners, and multi-agent frameworks that coordinate LLM calls through a central Python process. In most of these, the number of operations required to relate signals from two distant workers still grows with the distance between them, or coordination is funneled through a single orchestrating process that must itself parse, buffer, and re-emit every message — re-introducing a recurrent bottleneck one level up.

Self-attention, an attention mechanism relating different positions of a single sequence, has been used successfully to replace recurrence outright. To the best of our knowledge the Tmux Orchestration Architecture is the first coordination substrate to rely entirely on a terminal multiplexer — a program whose original purpose was to let *one human* keep *many* shells alive across disconnects — to instead let *one agent* keep *many agents* alive, addressable, and mutually routable, with the human as an optional attached client rather than a mandatory relay. Two properties of tmux make this possible and distinguish it from a plain subprocess pool:

- **Client/server separation.** A tmux *server* owns the sessions; *clients* merely attach to view and drive them. Detaching a client does not kill the work. The swarm's state is durable with respect to the human's presence — the human may detach, sleep, and reattach, and the computation persists. This is the substrate analog of a residual connection carrying state around each interaction (§3.1).
- **Programmatic control.** Beyond the interactive UI, tmux exposes a fully scriptable command surface and a machine-readable *control mode* (`-C`/`-CC`) in which every event is reported as a structured, `%`-prefixed message on a stream [7]. An agent, not just a human, can therefore be a first-class tmux client.

3 Model Architecture

Most competitive orchestration schemes have an encoder–decoder structure. Here, the *encoder* is the set of **worker agents** that map a task specification to a representation (a working tree, a diff, a set of captured outputs). The *decoder* is the **orchestrator agent** that, given those representations, generates the next instructions one at a time, autoregressively consuming its own prior decisions. The Tmux Orchestration Architecture follows this overall structure using stacked panes, content- and address-based attention, and per-agent feed-forward reasoning, shown schematically in Figure 1.

3.1 Session, Window, and Pane Stacks

The encoder and decoder are composed of a stack of identical structural layers, provided by tmux's three-level hierarchy:

- **Server** — one per socket (`-L name` / `-S /path/socket`), a namespace holding all sessions. Distinct sockets give fully isolated swarms that cannot address one another, useful for sandboxing untrusted sub-swarms.
- **Session** — a durable collection of windows, detachable from any client. One session per project or major objective.
- **Window** — a full-screen layer within a session, tiled into panes. One window per orchestration *head* (§3.2.2) or per phase.
- **Pane** — a single pseudo-terminal running exactly one agent process. The pane is the atomic unit: the residence of one Claude Code or Codex session.

Each worker pane is wrapped by two connections analogous to the residual connection and layer normalization around each Transformer sub-layer. A *residual path*: the pane's scrollback and, optionally, an append-only log via `pipe-pane -o -t <pane> 'cat » log'`, which streams output to a file for the pane's lifetime, so context flows *around* each interaction: LayerOutput $=$ Agent$(x)+x$, where x is the durable prior context. And a *normalization gate*: a checkpoint at which the orchestrator or human inspects and, if necessary, constrains the pane's output before it propagates — the human-in-the-loop gate of §5.4, bounding an agent's divergence before it affects shared state. The output of each sub-layer is thus Gate$(x+\text{Agent}(x))$, with Agent implemented by the LLM inside the pane and Gate by the review policy.

3.2 Attention

An orchestration attention function maps a *query* (the orchestrator's current subgoal) and a set of *key–value* pairs (the addressable worker panes and their captured contents) to an output (the next control action). Let the swarm be a set of panes $P=\{p_1,\ldots,p_n\}$. Each pane exposes, via tmux *format variables*, a **key** k_i (its addressable, queryable metadata) and a **value** v_i (its current content):

```
k_i = ( #{session_name}, #{window_index}, #{pane_index},
        #{pane_current_command}, #{pane_title}, #{@role},
        #{pane_dead}, #{window_silence_flag} )
v_i = capture-pane -p -t "#{session_name}:#{window_index}.#{pane_index}"
```

The keys are cheaply enumerable for the whole swarm in a single call:

```
tmux list-panes -a -F '#{session_name}:#{window_index}.#{pane_index} |
  cmd=#{pane_current_command} role=#{@role} dead=#{pane_dead}'
```

3.2.1 Scaled Dot-Product Attention (Scaled Selection)

We call our particular attention "Scaled Selection." The input consists of a query q (the current subgoal), keys k_i of dimension d_k (pane metadata), and values v_i (pane contents). We compute a relevance score between the query and each pane's key, apply a softmax to obtain weights, and read the weighted values:

$$\text{OrchAttention}(q,K,V)=\sum_i \text{softmax}_i\left(\frac{\text{score}(q,k_i)}{\sqrt{d_k}}\right)\cdot\text{Capture}(p_i). \tag{1}$$

In practice score(q,k_i) is realized as a *format filter* — a predicate over the pane's key fields that the orchestrator evaluates with `list-panes -f` / `if-shell -F`. For example, "attend to reviewer workers currently blocked at a shell prompt":

```
tmux list-panes -a \
  -f '#{&&:#{==:#{@role},reviewer},#{==:#{pane_current_command},bash}}' |
  -F '#{session_name}:#{window_index}.#{pane_index}'
```

The scaling factor $1/\sqrt{d_k}$ has a concrete operational meaning. As the swarm grows, raw relevance scores over many panes tend to produce a diffuse selection — the orchestrator tries to attend to too many workers at once, its control bandwidth saturates, and the softmax pushes into regions of vanishing marginal attention per agent (the human, watching, experiences this as thrash). We counteract this by scaling down relevance with swarm breadth ($d_k \approx$ the number of distinguishing key fields active at the current depth), keeping *fan-out bounded*: each step reads and writes a small, sharp

set of panes rather than a blurred average of all of them. Empirically, top-k (hard) selection — `head -k` on the filtered pane list — is the sparse limit of this softmax and is what we use in deployment. Reading a value is `capture-pane`; writing to the selected panes is `send-keys`, with a paste-buffer path for large or special-character payloads:

```
tmux capture-pane -p -t "$target" -S -200         # READ  (the "V")
tmux send-keys    -t "$target" "run the tests" Enter   # WRITE (route subgoal)
tmux load-buffer  -b task ./subtask-prompt.txt    # large payload, no escaping...
tmux paste-buffer -b task -t "$target"            # ...injected into stdin
```

3.2.2 Multi-Head Orchestration

Rather than run a single orchestration channel over the full swarm, we found it beneficial to run h channels in parallel — one per tmux **window** — each with its own selection projection (its own `@role` filter, subset of the swarm, and subgoal). Each head i attends independently,

$$\text{head}_i = \text{OrchAttention}\left(qW_i^Q, KW_i^K, VW_i^V\right), \tag{2}$$

where the projections W_i are, concretely, the per-window scoping of which panes and key fields that head considers relevant. Multi-head orchestration lets the architecture jointly attend to different parts of the project at different phases: head 1 supervises implementation panes, head 2 a test/CI pane, head 3 documentation, head 4 a long-running build. The heads run truly concurrently because each pane is an independent OS process; there is no time-slicing of a single operator. The heads are combined by concatenation into a shared **blackboard** — persistent key/value state in tmux user-options, readable and writable by any head:

```
tmux set-option  -g @blackboard/build_status "green"   # write  (Concat)
tmux show-option -gv @blackboard/build_status          # read
```

$\text{MultiHead} = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\, W^O$, where W^O is the reconciliation policy that resolves conflicting writes (last-writer-wins, or an orchestrator-mediated merge).

3.2.3 Applications of Attention in Our Model

The architecture uses orchestration attention in three ways. **(1) Orchestrator→worker cross-attention:** the decoder queries over all worker panes, reading their captured state and writing their next instruction — the analog of encoder–decoder attention, at $O(1)$ path length via direct pane addressing. **(2) Worker self-attention:** workers attend to one another *without* routing through the orchestrator when a direct pipe is authorized, `capture-pane -p -t A | ... | send-keys -t B`. **(3) Masked orchestrator self-attention:** the orchestrator attends to its own prior decisions (scrollback, blackboard) to remain autoregressive, but is *masked* from depending on subtasks that have not yet completed. The mask is enforced with `wait-for`: a barrier that blocks the orchestrator from consuming a result until the producing worker signals completion (§3.3), preserving the auto-regressive property.

3.3 Position-wise Feed-Forward Networks (Per-Agent Reasoning and Barriers)

In addition to attention sub-layers, each pane contains a fully-connected feed-forward network applied to that position independently: the LLM inference performed *inside* the agent. This is the actual "thinking" — identical in form at every pane but with different, per-pane parameters (the agent's own context window and system prompt). Synchronization between the attention layer and these per-agent computations is provided by `wait-for`, tmux's built-in barrier/signal channel:

```
tmux wait-for -S subtask-42   # worker: signal completion (last line of its task)
tmux wait-for    subtask-42   # orchestrator/dependent: block until signalled
```

`wait-for -L` / `-U` additionally provide a lock/unlock primitive for mutual exclusion over the shared working tree, preventing two workers from writing the same files concurrently.

3.4 Embeddings and Readout

Before a subtask enters the swarm it is *embedded* into the substrate: a natural language objective is tokenized into a shell-injectable instruction and a target address (which pane, which role). Symmetrically, the *readout* projects a pane's

Table 1: Maximum path length, per-step control operations, and sequential human interventions for different orchestration substrates. n is the number of agents in the swarm; k the fan-out of a single orchestration step ($k \ll n$).

Substrate	Control ops / step	Sequential (human) ops	Max inter-agent path
Recurrent human management	$O(n)$	$O(n)$	$O(n)$
Central-process orchestrator	$O(n)$	$O(1)$	$O(n)$ through the process
Tmux orchestration (this work)	$O(k)$	$O(1)$, at the gate only	$O(1)$

raw terminal output back into a decision-usable representation via `capture-pane` (with `-e` to retain escape sequences or `-J` to rejoin wrapped lines). As in the Transformer, we share the same learned mapping between the input embedding (how we phrase instructions to agents) and the pre-readout transformation (how we parse their replies): the orchestrator's prompt conventions and its output parser are two directions of one shared protocol, which for control-mode clients is the `%`-prefixed structured stream itself [7].

3.5 Positional Encoding

The swarm, as described, is a *set* of panes — orchestration attention is permutation-invariant and by itself carries no notion of which agent came first or which subtask depends on which. Since dependency order matters, we must inject information about the position of each agent in the topology and in time.

Tmux supplies positional structure natively. Each pane has a **discrete spatial coordinate** $(session, window\_index, pane\_index)$ that is stable for the pane's lifetime and totally ordered under `base-index`/`pane-base-index`. We combine this with a **temporal coordinate** from event hooks and the `#{t:...}` time formats, so each agent carries a position signal analogous to the sinusoidal encoding, letting the orchestrator reason about dependency order without a recurrent scan:

$$\mathrm{PE}(\text{pane}) = \big(window\_index \cdot B + pane\_index,\ \ timestamp\ of\ last\ state\ change\big).$$

The temporal component is maintained *event-drivenly* by hooks rather than by polling:

```
# Stamp a pane's position in time whenever it goes quiet (likely: finished a step)
tmux set-hook -g alert-silence \
  'set-option -p @last_silent "#{t:#{now}}" ; run-shell "notify-orch #{pane_id}"'
# React to a worker crashing (a "position" removed from the sequence)
tmux set-hook -g pane-died \
  'run-shell "orch handle-death #{pane_id} status=#{pane_dead_status}"'
```

Available hook events include `pane-died`, `alert-activity`, `alert-silence`, `alert-bell`, `session-created`, `client-attached`, and `client-detached` [8]. Activity/silence monitoring is enabled per window with `monitor-activity on` and `monitor-silence <seconds>`; a pane silent for s seconds is, with high probability, a worker that has finished its step and is waiting — precisely the event the orchestrator must attend to next. This gives the swarm a *positional* sense of "who just spoke and who just went quiet," computed once and event-drivenly, rather than re-derived by a sequential human sweep.

4 Why Tmux

In this section we compare the tmux orchestration substrate to alternatives — recurrent human management and centralized-process orchestration — on three desiderata: the total control cost per coordination step; the amount of computation that *must be sequential*, measured as unavoidable human interventions in the inner loop; and the maximum path length between any two agents that must exchange information. Shorter paths make it easier to route a result produced deep in the swarm into the context of a distant consumer.

As noted in Table 1, a self-attending orchestrator connects any two agents in a constant number of addressable operations (`capture-pane -t A | ... | send-keys -t B`), whereas recurrent human management requires the human to personally relay every hop, serializing the whole swarm through one mind. A central-process orchestrator removes the human from each hop but re-introduces an $O(n)$ bottleneck at the process that must parse and re-emit every message; tmux avoids this because the *substrate itself* is the router — panes are directly addressable and can be wired worker-to-worker without a relay.

Two further benefits motivated the choice of tmux. **Durability/detachability:** because the server outlives its clients, the human can detach entirely and the swarm keeps running; work is not lost on disconnect. This is what lets the human be

in the loop *eventually* (§5.4) rather than *always*. **Recursion:** a pane may run a tmux client that owns its own server or session, so an orchestrator can spawn *sub-orchestrators*, each managing a sub-swarm; the addressing composes via nested prefixes (`C-b C-b ...`) and per-socket namespaces. Depth-d recursion yields a coordination *tree*; the human attaches only at the root and can literally `choose-tree` to watch the whole hierarchy.

5 Deployment

This section describes how a swarm is decomposed, batched, scheduled, and regularized in operation — the "training regime" of the architecture.

5.1 Task Decomposition and Batching

Each engineering objective is decomposed by the orchestrator into subtasks batched by *independence*: subtasks with no shared file dependency are dispatched to distinct worker panes in the same step and proceed in parallel; dependent subtasks are serialized behind `wait-for` barriers (§3.3) and `wait-for -L` locks over shared paths. Batches are sized to keep fan-out k bounded (§3.2.1) so neither the orchestrator's control bandwidth nor the human's review bandwidth saturates.

5.2 Hardware and Schedule

A swarm runs on a single host (or a `tmux -S` socket shared over SSH), one pane per worker process. The orchestrator loop alternates between an *event-driven* phase — blocking on hooks and `wait-for` signals, consuming near-zero CPU while workers think — and a *dispatch* phase triggered when a pane goes silent, dies, or signals completion. Control mode (`-CC`) is used when the orchestrator is itself an agent: it reads the `%`-prefixed notification stream (`%output`, `%window-pane-changed`, `%exit`, ...) directly rather than screen-scraping [7].

5.3 Optimizer (the Control Loop)

The orchestration policy is the "optimizer" driving the swarm toward the objective. We use an event-driven loop with bounded polling as a fallback:

```
loop:
  wait on { hooks(pane-died, alert-silence), wait-for signals }   # event-driven
  on signal from pane p:
     v <- capture-pane(p)                      # attend (read)
     decision <- orchestrator.step(q, v)       # per-agent FFN of the decoder
     apply(decision):                          # attend (write)
        send-keys / paste-buffer to selected panes   # continue work
        respawn-pane p                                # restart a crashed worker
        break-pane / join-pane / swap-pane            # reshape topology
        set-option @blackboard/...                    # update shared state
     if decision.needs_human: escalate_to_gate()      # see 5.4
```

We adopt a schedule analogous to warmup-then-decay: early in an objective the orchestrator polls more aggressively and escalates to the human more readily (*warmup* — establishing shared context and trust); as the objective stabilizes it relies more on event-driven hooks and widens the human-gate interval (*decay*). `respawn-pane`/`respawn-window` provide restart-on-failure, and `break-pane`/`join-pane` let the optimizer restructure the swarm topology mid-run.

5.4 Regularization (Human-in-the-Loop Gate)

We employ several regularizers to prevent the swarm from overfitting to a locally plausible but globally wrong plan. **Human-gate:** the single most important regularizer. Before any batch of decisions with irreversible or outward-facing effects (a push, a deploy, a destructive command) propagates, it is surfaced to the attached human via `display-popup`, `confirm-before`, or a `choose-tree` review, and blocked on `wait-for` until acknowledged. The human is thus in the loop *eventually and where it matters*, not in every inner iteration — the analog of applying regularization at the layer boundaries rather than to every activation. **Silence dropout:** panes silent beyond a threshold are treated as dropped for the current step (`monitor-silence`), forcing the orchestrator not to rely on any single worker always being available.

Table 2: Variations on the Tmux Orchestration Architecture. h = orchestration heads (windows), k = per-step fan-out, d = recursion depth.

	h	k	d	**Human gate**	**Notes**
base	3	4	1	on irreversible ops	balanced; recommended default
(A) single-head	1	4	1	same	simpler; loses phase separation
(B) wide fan-out	3	12	1	same	saturates review bandwidth; thrash
(C) deep recursion	3	4	3	root only	powerful; hardest to supervise
(D) no gate	3	4	1	**off**	fastest and least safe; not advised

Checkpoint/residual: `pipe-pane` logs and periodic git commits provide the residual path that lets the swarm recover context after a crash or a bad step.

6 Results

We report qualitative and illustrative operational results; a rigorous benchmark of swarm throughput across task suites is left to future work, and the figures below are illustrative of the regime rather than a controlled measurement.

Orchestration throughput. Replacing recurrent human relay with $O(1)$ pane addressing removes the human from the inner loop, so wall-clock throughput scales with *available parallel workers* rather than *human cycle time*, up to the point where review bandwidth at the gate (§5.4) becomes the binding constraint. A single operator thus supervises a swarm whose aggregate work rate is set by k concurrent agents rather than by one.

Architecture variations. Table 2 varies the number of heads h, per-step fan-out k, recursion depth d, and the human gate. Removing the scaling discipline on fan-out (row B) degrades supervisory quality — the operator cannot review 12 parallel decision streams — and removing the human gate entirely (row D) maximizes raw speed while forfeiting the regularization that keeps the swarm aligned with intent. Deep recursion (row C) is the most capable configuration but concentrates oversight at the root, recommended only once trust in the lower levels is established.

Generalization to heterogeneous agents. Because the substrate addresses *panes*, not model APIs, the architecture generalizes across agent types without modification: a pane may hold Claude Code, a Codex session, a plain REPL, or a build process, all attended to through the same `send-keys`/`capture-pane` interface. The orchestrator need not know what runs inside a pane, only its address and its `@role` — the substrate is model-agnostic.

7 Conclusion

We presented the Tmux Orchestration Architecture, a model for supervising coding-agent swarms based entirely on the terminal multiplexer, replacing recurrent human management with direct, addressable attention between agents. For orchestration tasks, a single agent driving a tmux swarm connects any two workers in $O(1)$ operations, removes the human from the inner loop, and — by virtue of the server outliving its clients and panes being able to run tmux themselves — supports detachable, recursive, semi-autonomous operation with the human retained as a supervisory gate. We plan to apply it to swarms larger than a single host (federating over `tmux -S` sockets and SSH), to learned selection policies that replace hand-written format filters, and to tighter human-gate ergonomics. We make one claim, plainly: for keeping many coding agents alive, addressable, and mutually routable under one human's supervision — **tmux is all you need**.

Acknowledgements. To every worker pane that went silent at exactly the right moment, and to the human who stayed in the loop, eventually.

References

[1] D. Bahdanau, K. Cho, Y. Bengio. *Neural Machine Translation by Jointly Learning to Align and Translate.* ICLR, 2015.

[2] S. Hochreiter, J. Schmidhuber. *Long Short-Term Memory.* Neural Computation, 1997.

[3] I. Sutskever, O. Vinyals, Q. V. Le. *Sequence to Sequence Learning with Neural Networks.* NeurIPS, 2014.

[4] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, I. Polosukhin. *Attention Is All You Need.* NeurIPS, 2017.

[5] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, Y. Cao. *ReAct: Synergizing Reasoning and Acting in Language Models.* ICLR, 2023.

[6] N. Marriott et al. *tmux — terminal multiplexer, tmux(1) manual page.* `https://man7.org/linux/man-pages/man1/tmux.1.html`

[7] tmux project. *Control Mode.* tmux/tmux Wiki. `https://github.com/tmux/tmux/wiki/Control-Mode`

[8] tmux project. *Hooks and Notifications.* tmux/tmux documentation.

A Minimal Reproducible Swarm

A complete two-worker swarm with an orchestrator, a human gate, and event-driven scheduling, expressed entirely in tmux.

```
#!/usr/bin/env bash
set -euo pipefail
SOCK="swarm"                        # isolated server namespace: tmux -L "$SOCK"
T() { tmux -L "$SOCK" "$@"; }

# 1. Create the durable, detached session (survives human detach).
T new-session -d -s proj -n impl

# 2. Spawn two worker panes, each an independent coding agent, tagged by role.
T send-keys -t proj:impl.0 'claude' Enter ; T set-option -p -t proj:impl.0 @role worker
T split-window -t proj:impl -h
T send-keys -t proj:impl.1 'codex'  Enter ; T set-option -p -t proj:impl.1 @role worker

# 3. A dedicated head (window) for the orchestrator.
T new-window -t proj -n orch
T set-option -p -t proj:orch.0 @role orchestrator

# 4. Residual logging for every worker pane.
for p in proj:impl.0 proj:impl.1; do
  T pipe-pane -o -t "$p" "cat >> logs/${p//[:.]/_}.log"
done

# 5. Positional / event encoding: react when a worker goes quiet or dies.
T set-hook -g alert-silence 'run-shell "orchestrator wake #{pane_id}"'
T set-hook -g pane-died     'run-shell "orchestrator handle-death #{pane_id} #{pane_dead_status}"'
for p in proj:impl.0 proj:impl.1; do
  T set-option -t "${p%.*}" monitor-silence 20     # silence => finished a step
done

# 6. Attend: read a worker, decide, route the next subtask (the control loop).
read_pane() { T capture-pane -p -t "$1" -S -200; }
route()     { T send-keys -t "$1" "$2" Enter; }
gate()      { T display-popup -t proj -E "orchestrator review --hold $1"; }  # human

# 7. Human attaches at the root to supervise; detaching leaves the swarm running.
#    tmux -L swarm attach -t proj      # choose-tree (C-b s / C-b w) to watch heads
```